

## WEEK 2

| #  | Topic                                                  | Type                   |
|----|--------------------------------------------------------|------------------------|
| Q1 | Text statistics (term frequency, dictionary, plotting) | Python coding          |
| Q2 | Stemming - TRUE/FALSE                                  | Concept / short answer |
| Q3 | N-grams (bigrams & trigrams)                           | Python coding          |
| Q4 | HTML tags & hyperlink extraction                       | Python coding + ethics |

### Question 1 - Text Statistics

#### Task summary (from the PDF):

1. Open an XML file and represent it as a list of sentences/paragraphs (remove <p>, </p>, \n)
2. Find all terms with length > 2 and count their frequencies → store in a dictionary doc
3. Print the total number of terms (e.g., 137)
4. Print the top-10 terms in descending order
5. Plot the distribution of top-10 terms

#### Key concepts used

- `readlines()` - reads file into a list of lines
- `.replace()` - strips XML tags and newlines
- `.split()` - tokenises each line into terms
- `.lower()` - normalises case
- `try/except KeyError` - builds frequency dictionary
- `sorted(..., key=lambda x:x[1], reverse=True)` - sorts by frequency descending
- `zip(*myList)` - unpacks list of tuples into x, y for plotting
- `plt.plot(x, y)` - line plot

#### Solution steps (as shown in the PDF)

##### Step 1 - Read and clean the file:

```
myfile = open('6146.xml')
file_ = myfile.readlines()
```

```
doc = {}
word_count = 0
```

```
text = [line.replace("<p>", "").replace("</p>",
""),
        line.replace("\n", "")
        for line in file_ if line.startswith('<p>')]
```

##### Step 2 - Count term frequencies (length > 2):

```
for line in text:
    for term in line.split():
        word_count += 1
        term = term.lower()
        if len(term) > 2:
            try:
                doc[term] += 1
            except KeyError:
                doc[term] = 1
```

```
print(word_count) # e.g., 137
```

##### Step 3 - Sort and get top-10:

```
myList = doc.items()
myList = sorted(myList, key=lambda x: x[1],
reverse=True) # sorted by value
myList = list(myList)
myList = myList[:10]
print(myList)
x, y = zip(*myList)
```

##### Step 4 - Plot:

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.show()
```

#### Expected output:

```
[('the', 8), ('technical', 2), ('bounce', 2), ('said',
2),
 ('and', 2), ('not', 2), ('due', 2), ('rose', 2),
 ('argentina's', 2), ('argentine', 1)]
```

### Q2 - Stemming: Key Concepts (from the PDF)

#### Two types of stemmers:

##### 1. Algorithmic stemmer

- Uses a **small program** to decide whether two words are related
- Based on knowledge of **word suffixes** for a particular language

##### 2. Dictionary-based stemmer

- Has **no logic of its own**
- Instead relies on **pre-created dictionaries** of related terms to store term relationships

#### What stemming does:

- Captures relationships between **different variations** of a word
- Reduces different word forms from:
  - **Inflection** (e.g., plurals, tenses)
  - **Derivation** (e.g., making a verb into a noun by adding suffixation)
- Reduces them all to a **common stem**

#### Effect on search:

- Using a stemmer for English search produces a **small but noticeable improvement** in quality of results

The key thing to remember is the **mix-up between the two stemmer types** - that's exactly what Q2 tests.

### Question 3 - N-grams: Key Concepts (from the PDF)

Sure! Let me break down each part in more detail.

#### 1. What are N-grams?

Think of a **sliding window** moving one word at a time:

**Document:** "Tropical fish are"

##### Bigrams (window of 2):

- "Tropical fish"
- "fish are"

##### Trigrams (window of 3):

- "Tropical fish are"

The window **overlaps** - each step moves only **one word** forward.

#### 2. Building bigrams and trigrams

```
words = line.split()
stems = words
bigrams = [stems[i]+' '+stems[i+1] for i in
range(len(stems)-1)]
```

- `range(len(stems)-1)` - stops one short so `i+1` doesn't go out of bounds
- `range(len(stems)-2)` - stops two short for trigrams so `i+2` doesn't go out of bounds

#### 3. The `is_highlight()` function

```
def is_highlight(gram, keywords):
    x = False
    for y in keywords:
        if y in gram:
            x = True
    return x
```

- Checks if **any keyword** appears in the gram
- Returns `True` if at least one keyword is found
- Returns `False` if none found

#### 4. Removing punctuation

```
line = line.translate(str.maketrans('', '',
string.digits))\
```

```
.translate(str.maketrans(string.punctuation, '
'*len(string.punctuation)))
```

- First `.translate()` - removes all **digits**
- Second `.translate()` - replaces all **punctuation** with spaces
- This ensures words like "aquariums." don't get treated differently from "aquariums"

## Question 4 - HTML Tags & Hyperlink Extraction

### Two parts:

- (a) Python program to extract all hyperlinks from an HTML file using `HTMLParser`
- (b) Ethical issues of extracting hyperlinks from the Web

### Part (a) - How the code works

**Key idea:** Hyperlinks in HTML look like this:

```
<a href="/wiki/Fish">Fish</a>
```

- The tag is `<a>` (anchor tag)
- The hyperlink is stored in the `href` attribute

**The solution uses `HTMLParser` to detect start tags:**

```
from html.parser import HTMLParser
```

```
class Parser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        global start_tags
        global attrs_names
        if tag == 'a':
            start_tags.append(tag)
            attrs_names.append(attrs)
```

```
start_tags = []
attrs_names = []
parser = Parser()
```

```
# Open and feed the HTML file
myHtml = open('html_example.html')
file_html = myHtml.read()
parser.feed(file_html)
```

```
# Extract destination links
hyper_links = [x[0][1] for x in attrs_names]
print(hyper_links)
myHtml.close()
```

### Key steps:

1. Create a class that inherits from `HTMLParser`
2. Override `handle_starttag()` - called every time a start tag is found
3. Check if `tag == 'a'` - only process anchor tags
4. Store the tag and its attributes
5. Extract the link: `x[0][1]` - first attribute's value (the `href`)

### Part (b) - Ethical Issues (from the PDF)

#### Two main issues:

##### 1. Denial of Service (DoS)

- A web crawler sends many requests to a server repeatedly
- Each request consumes server resources
- Too many requests in a short time can **prevent normal users** from accessing the site
- This looks like a **DoS attack**

### 2. Legal issues

- If the website's **terms and conditions** forbid downloading/copying content
- Extracting content would be a **legal/ethical violation**
- The PDF recommends seeking advice from a professional with knowledge of **intellectual property (copyright) legislation**

## WEEK 3

| #  | Topic                                                 | Type          |
|----|-------------------------------------------------------|---------------|
| Q1 | Retrieval models (Boolean, Vector Space) - TRUE/FALSE | Concept       |
| Q2 | Bayes Classifier - calculate $P(D R)$                 | Calculation   |
| Q3 | Probabilistic Models - term weighting formula         | Short answer  |
| Q4 | Document features & ranking - TRUE/FALSE              | Concept       |
| Q5 | Abstract Model of Ranking - $f(A,B,Q)$                | Concept       |
| Q6 | Word Embedding - Word2Vec in Python                   | Python coding |
| Q7 | Word Embedding - BERT (Optional)                      | Python coding |

## Q1 - Retrieval Models: Key Concepts (from the PDF)

### Four statements to understand

#### (1) Boolean Retrieval Model

- Also known as **exact-match retrieval**
- Documents are retrieved **only if they exactly match** the query specification
- Otherwise they are **not retrieved** at all
- Relevance is **explicit**.
- **Binary** outcome (True or False)

#### (2) Searching by Numbers

- Developing queries with a focus on the **size of the retrieved set**
- Called "**searching by numbers**"
- This is a **consequence of the limitations** of the Boolean retrieval model

#### (3) Vector Space Model

- Documents and queries are assumed to be part of a **t-dimensional vector space**
- Where **t = number of index terms** (words, stems, phrases, etc.)

#### (4) Relevance in Vector Space Model - THIS IS THE FALSE ONE

- **implicit assumption** that relevance is related to the **similarity of query and document vectors**

### The key thing to remember:

- Vector Space Model → relevance is **implicit** (similarity-based), **not explicit**

## Q2 - Bayes Classifier: Key Concepts (from the PDF)

### The problem:

- Query  $Q = \{US, ECONOM, ESPIONAG\}$
- Collection  $C = \{D1, D2, D3, D4, D5, D6, D7\}$
- Goal: calculate  $P(D|R)$  - the probability that query  $D$  occurs in relevant documents

### Step 1 - Define the full term set T and binary vector of D

All unique terms:  $T = \{US, BILL, ECONOM, ESPIONAG, GERMAN, MAN, VW, SPY\} = \{d1, d2, d3, d4, d5, d6, d7, d8\}$

Query  $D = \{US, ECONOM, ESPIONAG\} = \{d1, d3, d4\}$

Binary vector of  $D = (1, 0, 1, 1, 0, 0, 0, 0)$

- 1 = term is in D, 0 = term is not in D

### Step 2 - Identify relevant documents

From the table, relevant documents (relevance = yes) are: **D2, D3, D4**

Their binary vectors:

- $D2 = (1, 0, 1, 0, 0, 0, 0, 1)$
- $D3 = (1, 1, 1, 1, 0, 0, 0, 0)$
- $D4 = (1, 1, 1, 1, 0, 0, 0, 0)$

### Step 3 - Calculate $pi = P(d_i|R)$ for each term in D

$p1 = P(US|R)$  = how many relevant docs contain US / total relevant docs =  $3/3 = 1.0$

$p3 = P(ECONOM|R) = 3/3 = 1.0$

$p4 = P(ESPIONAG|R) = 2/3 = 0.667$

### Step 4 - Apply the formula

$$P(D|R) = \prod_{i=1}^t P(d_i|R)$$

$P(D|R) = P(d1|R) \times P(d3|R) \times P(d4|R) = 1 \times 1 \times 0.667 = 0.667$

### Key concept to remember:

- $pi = P(d_i|R)$  = number of times term  $d_i$  appears in relevant documents / total number of relevant documents
- Terms are assumed to be **independent** → multiply probabilities together

## Q3 - Probabilistic Models Summary

Formula:

$$w_1(t) = \log \left[ \frac{r(t)/R}{n(t)/N} \right]$$

What each variable means:

| Variable | Meaning                                               |
|----------|-------------------------------------------------------|
| N        | Total number of documents                             |
| R        | Total number of relevant documents                    |
| r(t)     | Number of <b>relevant</b> documents containing term t |
| n(t)     | Number of <b>all</b> documents containing term t      |

How to calculate:

1. Count **r(t)** - relevant docs with term t
2. Count **n(t)** - all docs with term t
3. Divide:  $r(t)/R$  and  $n(t)/N$
4. Divide the two ratios → apply **log**

Key interpretation:

- **High  $w_1(t)$**  → term appears more in relevant docs → **more useful** for retrieval
- **Low  $w_1(t)$**  → term is common across all docs → **less useful** for retrieval

## Q4 - Document Features & Ranking: Key Concepts (from the PDF)

Four statements to understand:

### (1) Document Feature

- A document feature is some **attribute of the document** we can express **numerically**

### (2) Ranking Function

- Takes data from **document features combined with the query**
- Produces a **score**

### (3) Types of Document Features - THIS IS THE FALSE ONE

- Statement says: "Topical features are the **only** features we can find in documents"
- This is **FALSE** - documents also have **Quality features**
- Example of quality feature: **meta information** such as **days since last update** of the document

### (4) Score Interpretation

- **High score** → system thinks document is a **good match** for the query
- **Low score** → system thinks document is a **poor match** for the query

Key thing to remember:

- There are **two types** of document features:
  - **Topical features** - about the content/topic of the document
  - **Quality features** - meta information e.g. days since last update

## Q5 - Abstract Model of Ranking: Key Concepts (from the PDF)

Two ranking functions to understand:

### R(Q, D) - Standard ranking function

- Compares **each document** with the query
- Computes a **score**
- Scores are used to determine the **final ranked list**

### f(A, B, Q) - Alternate ranking function

- Compares **two documents A and B** against query Q
- Returns:
  - **1** → if A should be ranked **higher** than B
  - **-1** → if A should be ranked **lower** than B
  - **0** → special case when A and B have the **same score**

The question asks: If you have  $R(Q, D)$ , how do you use it in a system that requires  $f(A, B, Q)$ ?

Solution from the PDF:

$$f(A, B, Q) = \begin{cases} 1 & \text{if } R(Q, A) > R(Q, B) \\ -1 & \text{if } R(Q, B) > R(Q, A) \end{cases}$$

- If  $R(Q, A) > R(Q, B) \rightarrow A$  scores higher  $\rightarrow f(A, B, Q) = 1$
- If  $R(Q, A) < R(Q, B) \rightarrow B$  scores higher  $\rightarrow f(A, B, Q) = -1$
- If  $R(Q, A) = R(Q, B) \rightarrow$  tie  $\rightarrow f(A, B, Q) = 0$  (or assign 1 or -1)

#### In plain English:

- Just compute  $R(Q, D)$  for both documents separately
- **Compare the two scores** to decide which document ranks higher
- That comparison result becomes the output of  $f(A, B, Q)$

## Q6 - Word Embedding (Word2Vec): Key Concepts (from the PDF)

### What is Word2Vec?

- Used to generate **distributed word vector representations** in Python
- Required modules: **NLTK** and **Gensim**

### Steps in the solution:

#### Step 1 - Read and clean the file:

```
sample = open("alice.txt")
s = sample.read()
f = s.replace("\n", " ") # replace newlines with space
data = []
```

#### Step 2 - Tokenize into sentences then words:

```
for i in sent_tokenize(f):
    temp = []
    for j in word_tokenize(i):
        temp.append(j.lower())
    data.append(temp)
```

- `sent_tokenize()` - splits text into **sentences**
- `word_tokenize()` - splits each sentence into **words**
- Result: `data = list of lists` where each inner list = words of one sentence

#### Step 3 - Train Word2Vec model (CBOW):

```
model1 = gensim.models.Word2Vec(data, min_count=1, vector_size=10, window=5)
```

- `min_count=1` - include all words
- `vector_size=10` - each word represented as a 10-dimensional vector
- `window=5` - context window size

#### Step 4 - Compute similarities:

```
model1.wv.similarity('alice', 'wonderland') # higher similarity ~0.945
model1.wv.similarity('alice', 'machines') # lower similarity ~0.546
```

### Key results from the PDF:

- Similarity between 'alice' and 'wonderland' = **0.945** (high - they appear in similar contexts)
- Similarity between 'alice' and 'machines' = **0.546** (lower - less related)

### Key concepts:

- **CBOW** - predicts a word from its context
- **Skip-gram** - predicts context from a word
- `wv.similarity()` - computes **cosine similarity** between two word vectors

- Higher similarity  $\rightarrow$  words appear in **more similar contexts**

## Q1 - Retrieval Models

- Boolean = exact-match, binary outcome
- Vector Space = t-dimensional, relevance is implicit (similarity-based)
- Searching by numbers = consequence of Boolean's limitations

## Q2 - Bayes Classifier

- Build binary vectors  $\rightarrow$  calculate  $P(d_i|R)$  for each query term  $\rightarrow$  multiply together
- $P(d_i|R) = \text{relevant docs containing term} / \text{total relevant docs}$

## Q3 - Probabilistic Models

- $wl(t) = \log[r(t)/R / n(t)/N]$
- $r(t)$  = relevant docs containing term  $t$
- $n(t)$  = all docs containing term  $t$

## Q4 - Document Features

- Two types: **Topical** and **Quality** features
- Ranking function takes features + query  $\rightarrow$  produces score

## Q5 - Abstract Model of Ranking

- $f(A, B, Q) = 1$  if  $R(Q,A) > R(Q,B)$ ,  $-1$  if  $R(Q,A) < R(Q,B)$

## Q6 - Word2Vec

- `sent_tokenize`  $\rightarrow$  `word_tokenize`  $\rightarrow$  train Word2Vec  $\rightarrow$  `wv.similarity()`

## WEEK 4

| #  | Topic                                            | Type                  |
|----|--------------------------------------------------|-----------------------|
| Q1 | Inverted Index - TRUE/FALSE                      | Concept               |
| Q2 | Query Processing - Term-at-a-time Algorithm      | Calculation           |
| Q3 | Markov Chain - transition matrix & probability   | Concept + Calculation |
| Q5 | LLM-based Information Extraction - prompt design | Concept               |

## Q1 - Inverted Index: Key Concepts (from the PDF)

### Four statements to understand:

#### (1) Why is it called "Inverted"?

- Normally we think of **words being part of documents**
- An inverted index **flips this** - documents are associated with words
- That's why it's called **"inverted"**

#### (2) Binary inverted index and phrase matching - THIS IS THE FALSE ONE

- A binary inverted index only stores **1 (contains term) or 0 (doesn't contain term)**
- It can tell you if a document contains "tropical" AND "fish" separately
- But it **cannot** tell you if the document contains the **exact phrase "tropical fish"**
- To find exact phrases you need to add **position information** to the index

#### (3) Postings and Pointers

- Each index term has its own **inverted list** holding relevant data for that term
- Each entry in the list is called a **posting**
- The part of a posting that refers to a specific document or location is called a **pointer**

#### (4) Extents

- An extent is a **contiguous region** of a document
- Extents can be represented using **word positions**

#### Key thing to remember:

- Binary index = can find documents containing individual terms
- Binary index = **cannot** find exact phrases → need **position information**

## Q2: Term-at-a-time Algorithm

Query Q = {freshwater, fish, aquarium, tropical}

#### Inverted lists:

- L1 - freshwater: 1:1, 4:1
- L2 - aquarium: 3:1
- L3 - fish: 1:2, 2:3, 3:2, 4:2
- L4 - tropical: 1:2, 2:2, 3:1

**The idea:** Go through **one list at a time** and add counts to each document's running score.

#### Step 1 - Process L1 (freshwater):

- Document 1 gets +1, Document 4 gets +1
- Partial scores: **1:1, 4:1**

#### Step 2 - Process L2 (aquarium):

- Document 3 gets +1 (new document)
- Partial scores: **1:1, 3:1, 4:1**

#### Step 3 - Process L3 (fish):

- Document 1 gets +2 → 1+2=3
- Document 2 gets +3 (new)
- Document 3 gets +2 → 1+2=3
- Document 4 gets +2 → 1+2=3
- Partial scores: **1:3, 2:3, 3:3, 4:3**

#### Step 4 - Process L4 (tropical):

- Document 1 gets +2 → 3+2=5
- Document 2 gets +2 → 3+2=5
- Document 3 gets +1 → 3+1=4
- Document 4 - tropical not in L4 → stays at 3
- **Final scores: 1:5, 2:5, 3:4, 4:3**

#### Key rules to remember:

- If document already has a score → **add** to it
- If document is new → **start** with that count
- Process lists **one at a time** in order

## Q3 - Markov Chain: Key Concepts (from the PDF)

What is a Markov Chain?

- A model that describes a sequence of possible **states**
- The next state depends **only on the current state** - not the history
- Each transition has a **probability**

#### The example from the PDF:

John is sad and can be in one of **3 states**:

- **Sleep**
- **Run**
- **Ice Cream**

#### Transition diagram probabilities:

| Current State | → Sleep | → Run | → Ice Cream |
|---------------|---------|-------|-------------|
| Sleep         | 0.2     | 0.6   | 0.2         |
| Run           | 0.1     | 0.6   | 0.3         |
| Ice Cream     | 0.2     | 0.7   | 0.1         |

**Key rule:** Each row must **add up to 1.0** (100%)

**Part (1) - Question:** If John spent the day **sleeping**, what is the probability he will go for a **run** next day?

**Answer:** Look at Sleep row → Run column = **0.6 = 60%**

#### Part (2) - Transition matrix in Python:

```
[[0.2, 0.6, 0.2], # Sleep row
 [0.1, 0.6, 0.3], # Run row
 [0.2, 0.7, 0.1]] # Ice Cream row
```

#### Key things to remember:

- Read the matrix **row = current state, column = next state**
- Each row adds up to **1.0**
- Probability of transition = value at [**current state**][**next state**]

## Q5 - LLM-based Information

### Extraction: Key Concepts (from the PDF)

#### What is the task?

- Use an **LLM prompt** to extract specific entities from an XML document
- Return the result as a **valid Python dictionary**

#### Entities to extract:

- **Itemid** - integer
- **Organization** - string
- **Location** - string
- **Date** - string
- **Copyright** - string

#### Expected output:

```
{
  "Itemid": 780718,
  "Organization": "Reuters Limited",
  "Location": "European Community",
  "Date": "1997-08-06",
  "Copyright": "(c) Reuters Limited 1997"
```



}

### The solution prompt from the PDF:

You are an information extraction system.  
Extract entities  
from the XML document, and return the  
result strictly in  
valid Python dictionary using this  
template:

```
{
  Itemid: [integer],
  Organization: [string],
  Location: [string],
  Date: [string],
  Copyright: [string]
}
XML document: [paste XML here]
```

### Key things to remember:

- The prompt must specify the **exact output format** (Python dictionary)
- Each field must specify its **data type** (integer, string)
- The word "**strictly**" is important - it forces the LLM to return only the dictionary, nothing else

### Key design principles of a good extraction prompt:

1. Define the **role** - "You are an information extraction system"
2. Specify the **task** - "Extract entities from the XML document"
3. Specify the **output format** - "return strictly in valid Python dictionary"
4. Provide a **template** with field names and data types
5. Provide the **input document**

Here's a quick summary of everything you covered:

#### Q1 - Inverted Index

- Inverted = words point to documents (not the other way)
- Binary index cannot find exact phrases → need **position information**
- Posting = one entry in the list (doc:count)
- Pointer = part of posting referring to a document/location

#### Q2 - Term-at-a-time Algorithm

- Process one inverted list at a time
- Add counts to each document's running score
- New document → initialise with count
- Existing document → add to current score

#### Q3 - Markov Chain

- 3 states: Sleep, Run, Ice Cream
- Read matrix: **row** = **current state**, **column** = **next state**
- Each row adds up to **1.0**

- Sleep → Run = **0.6 = 60%**

#### Q4 - Markov Chain Python (Optional)

- `numpy.random.choice` for random state sampling
- Sequence probability = multiply all transition probabilities

#### Q5 - LLM Information Extraction

- Define role → specify task → specify output format → provide template → provide input
- Use "**strictly**" to force exact dictionary output

## WEEK 5

| #  | Topic                                                 | Type          |
|----|-------------------------------------------------------|---------------|
| Q1 | Language Models - Jelinek-Mercer smoothing            | Calculation   |
| Q2 | Likelihood Model - Python implementation              | Python coding |
| Q3 | Evaluation - Query logs & pooling - TRUE/FALSE        | Concept       |
| Q4 | Effectiveness Metrics - Recall, Precision, F1, FP, FN | Calculation   |

### Q1 - Language Models: Jelinek-Mercer Smoothing

The formula:

$$p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

What each variable means:

| Variable           | Meaning                                                              |
|--------------------|----------------------------------------------------------------------|
| $\lambda$ (lambda) | Smoothing parameter (given as 0.4)                                   |
| $f_{q_i,D}$        | Number of times query term $q_i$ occurs in document $D$              |
| $ D $              | Total number of word occurrences in document $D$                     |
| $c_{q_i}$          | Number of times query term $q_i$ occurs in the entire collection $C$ |
| $ C $              | Total number of word occurrences in collection $C$                   |

Then the overall document score:

$$P(Q|D) = \prod_{i=1} p(q_i|D)$$

Meaning: **multiply** all individual term probabilities together.

**Before calculating, we need two counts:**

$|D2|$  = total words in  $D2 = \{US, US, ECONOM, SPY\} = 4$

$|C|$  = total words across ALL documents:

- $D1 = 2, D2 = 4, D3 = 4, D4 = 4, D5 = 4, D6 = 5$
- $|C| = 2+4+4+4+4+5 = 23$

**Collection counts ( $c_{q_i}$ ):**

- US appears in:  $D2(2) + D3(1) + D4(1) = 4$
- ECONOM appears in:  $D2(1) + D3(1) + D4(1) = 3$
- ESPIONAG appears in:  $D3(1) + D4(1) + D5(1) = 3$

**Calculating each term for  $D2, \lambda=0.4$ :**

**$P(US|D2)$ :**

- $f_{US,D2} = 2, |D2| = 4, c_{US} = 4, |C| = 23$
- $= (1-0.4) \times (2/4) + 0.4 \times (4/23)$
- $= 0.6 \times 0.5 + 0.4 \times 0.17$
- **= 0.368**

**$P(ECONOM|D2)$ :**

- $f_{\text{ECONOM},D2} = 1, |D2| = 4, c_{\text{ECONOM}} = 3, |C| = 23$
- $= 0.6 \times (1/4) + 0.4 \times (3/23)$
- $= 0.6 \times 0.25 + 0.4 \times 0.13$
- $= \mathbf{0.202}$

#### P(ESPIONAG|D2):

- $f_{\text{ESPIONAG},D2} = 0$  (not in D2!),  $|D2| = 4, c_{\text{ESPIONAG}} = 3, |C| = 23$
- $= 0.6 \times (0/4) + 0.4 \times (3/23)$
- $= 0 + 0.4 \times 0.13$
- $= \mathbf{0.052}$

#### Final P(Q|D2):

- $= 0.368 \times 0.202 \times 0.052$
- $= \mathbf{0.003865}$

#### Key thing to remember:

- Even if a term is **not in the document** ( $f_{q_i,D} = 0$ ), the formula still gives a small probability because of the  $\lambda \times c_{q_i}/|C|$  part - that's the purpose of smoothing!

### Q3 - Query Logs & Pooling: Key Concepts (from the PDF)

Four statements - one is FALSE:

#### (1) Pooling technique

- Used for **very large data collections**
- Selects **top-k results** from rankings obtained by different retrieval algorithms
- Results merged into a **pool**, duplicates removed
- Documents presented in **random order** to people doing relevance judgments

#### (2) Pooling limitation

- Produces a large number of relevance judgments per query
- BUT the list is **incomplete**
- For a **new retrieval algorithm** that didn't contribute to the original pool → this could be a **problem**

#### (3) Implicit relevance judgments

- Many user actions (e.g., **query log data**) can be considered **implicit relevance judgments**
- Main drawback: **not as precise** as explicit relevance judgments

#### (4) Query logs - THIS IS THE FALSE ONE

- Statement says: "A typical query log does **NOT** contain user identifier or session identifier because of privacy"
- This is **FALSE** - a typical query log **DOES contain**:
  - User identifier
  - User session identifier
  - Query terms
  - List of URLs of results
  - Their ranks on the result list
  - Whether they were clicked on
  - Timestamps
- Privacy is an issue when query logs are **shared or distributed for research**

#### Key thing to remember:

- Query logs **DO** contain user identifiers - privacy becomes an issue when they are **shared**, not just collected

### Q4 - Effectiveness Metrics: Key Concepts (from the PDF)

First, understand the 4 sets:

Set

Meaning

**A** All **relevant** documents (from Table 1,  $Rel=1$ )

**A'** All **non-relevant** documents (from Table 1,  $Rel=0$ )

**B** **Retrieved** documents (top-6 from Table 2)

**B'** **Not retrieved** documents (remaining from Table 2)

#### From the PDF:

- $A = \{110, 111, 113, 114, 116, 117, 119, 123\}$
- $A' = \{112, 115, 118, 120, 121, 122, 124\}$
- $B = \{111, 112, 113, 110, 114, 119\}$  (top-6 ranked)
- $B' = \{115, 122, 118, 116, 123, 121, 120, 117, 124\}$

$A \cap B$  = documents that are both **relevant AND retrieved** =  $\{110, 111, 113, 114, 119\} = \mathbf{5 \text{ documents}}$

#### The 4 metrics:

**Recall** = how many relevant docs were retrieved:

$$Recall = \frac{|A \cap B|}{|A|} = \frac{5}{8} = 62.5\%$$

**Precision** = how many retrieved docs are relevant:

$$Precision = \frac{|A \cap B|}{|B|} = \frac{5}{6} = 83.33\%$$

**False Positive (FP)** = non-relevant docs that were retrieved:

$$FP = \frac{|A' \cap B|}{|A'|} = \frac{|\{112\}|}{7} = \frac{1}{7}$$

**False Negative (FN)** = relevant docs that were NOT retrieved:

$$FN = \frac{|A \cap B'|}{|A|} = \frac{|\{116, 117, 123\}|}{8} = \frac{3}{8}$$

**F1 Measure** = balance between Recall and Precision:

$$F1 = \frac{2 \times Recall \times Precision}{Recall + Precision} = \frac{2 \times \frac{5}{8} \times \frac{5}{6}}{\frac{5}{8} + \frac{5}{6}} = \frac{10}{14} = 71.43\%$$

#### Key things to remember:

- **Recall** = found relevant / all relevant → measures **coverage**
- **Precision** = found relevant / all retrieved → measures **accuracy**

- **FP** = retrieved but not relevant (retrieved **non-relevant**)
- **FN** = relevant but not retrieved (missed relevant)
- **F1** = combines both Recall and Precision into one score

#### Q1 - Jelinek-Mercer Smoothing

- Formula:  $p(q_i|D) = (1-\lambda) \times f_{q_i,D}/|D| + \lambda \times c_{q_i,C}/|C|$
- Multiply all term probabilities  $\rightarrow P(Q|D)$
- Even if term not in document  $\rightarrow$  still gets small probability (smoothing!)
- $\lambda$  = smoothing parameter

#### Q2 - Likelihood Model

- Call `index_docs()` first  $\rightarrow$  then `likelihood_IR(I, Q)`
- Output = ranked list by likelihood score

#### Q3 - Query Logs & Pooling

- Query logs DO contain user identifiers, session IDs, timestamps
- Privacy issue arises when logs are **shared/distributed**
- Pooling = merge top-k results from multiple algorithms, remove duplicates

#### Q4 - Effectiveness Metrics

- **A** = relevant docs, **A'** = non-relevant docs
- **B** = retrieved docs, **B'** = not retrieved docs
- Recall =  $A \cap B / |A| \rightarrow$  coverage
- Precision =  $A \cap B / |B| \rightarrow$  accuracy
- FP =  $A' \cap B / |A'| \rightarrow$  retrieved but not relevant
- FN =  $A \cap B' / |A| \rightarrow$  relevant but missed
- F1 =  $2 \times \text{Recall} \times \text{Precision} / (\text{Recall} + \text{Precision})$

## WEEK 6

| #  | Topic                                     | Type                 |
|----|-------------------------------------------|----------------------|
| Q1 | Edit Distance & Stemming - TRUE/FALSE     | Concept              |
| Q2 | Noisy Channel Model - spelling correction | Concept              |
| Q3 | Precision at rank k & Average Precision   | Calculation          |
| Q4 | MAP & Standard Recall Levels              | Calculation + Python |

### Q1 - Edit Distance & Stemming: Key Concepts (from the PDF)

Four statements to understand:

#### (1) Stem Class

- A **stem class** is the group of words that will be transformed into the **same stem** by the stemming algorithm
- e.g. "run", "running", "runs" all map to stem "run"

#### (2) Edit Distance

- The most common measure for comparing words/strings
- Defined as the **number of operations** required to transform one word into another
- Also called **Levenshtein distance**

#### (3) Damerau-Levenshtein Distance - THIS IS THE FALSE ONE

- Statement says: counts the **maximum** number of insertions, deletions, substitutions, or transpositions

- This is **FALSE** - it counts the **minimum** number of operations required
- Operations: insertions, deletions, substitutions, transpositions of single characters

#### (4) Speeding up edit distance calculation

- Techniques to speed up comparisons:
  - Restrict to words starting with the **same letter** (spelling errors rarely change the first letter)
  - Restrict to words of **same or similar length** (spelling errors rarely change word length)
  - Restrict to words that **sound the same**

#### Key thing to remember:

- Damerau-Levenshtein = **minimum** not maximum number of operations

### Q3 - Precision at Rank k & Average Precision

The data (same as Week 5 Q4):

Ranked output of IR Model 1:

| Rank | Doc | Relevant? |
|------|-----|-----------|
| 1    | 111 | ✓         |
| 2    | 112 | ✗         |
| 3    | 113 | ✓         |
| 4    | 110 | ✓         |
| 5    | 114 | ✓         |
| 6    | 119 | ✓         |
| 7    | 115 | ✗         |
| 8    | 122 | ✗         |
| 9    | 118 | ✗         |
| 10   | 116 | ✓         |
| 11   | 123 | ✓         |
| 12   | 121 | ✗         |
| 13   | 120 | ✗         |
| 14   | 117 | ✓         |
| 15   | 124 | ✗         |

Total relevant docs  $|A| = 8$

#### Part (1) - Precision at rank k:

**Precision at rank 6:**

- Top 6 docs: {111, 112, 113, 110, 114, 119}
- Relevant in top 6: {111, 113, 110, 114, 119} = 5
- $P@6 = 5/6 = 83.33\%$

**Precision at rank 10:**

- Top 10 docs: {111, 112, 113, 110, 114, 119, 115, 122, 118, 116}
- Relevant in top 10: {111, 113, 110, 114, 119, 116} = 6
- $P@10 = 6/10 = 60\%$

#### Part (2) - Average Precision:

Only calculate precision at ranks where a relevant document appears:

| Rank | Doc | Relevant? | Precision at this rank |
|------|-----|-----------|------------------------|
| 1    | 111 | ✓         | $1/1 = 1.0$            |
| 2    | 112 | ✗         | -                      |
| 3    | 113 | ✓         | $2/3 = 0.667$          |



|    |     |   |                     |
|----|-----|---|---------------------|
| 4  | 110 | ✓ | 3/4 = <b>0.75</b>   |
| 5  | 114 | ✓ | 4/5 = <b>0.8</b>    |
| 6  | 119 | ✓ | 5/6 = <b>0.833</b>  |
| 7  | 115 | X | -                   |
| 8  | 122 | X | -                   |
| 9  | 118 | X | -                   |
| 10 | 116 | ✓ | 6/10 = <b>0.6</b>   |
| 11 | 123 | ✓ | 7/11 = <b>0.636</b> |
| 12 | 121 | X | -                   |
| 13 | 120 | X | -                   |
| 14 | 117 | ✓ | 8/14 = <b>0.571</b> |
| 15 | 124 | X | -                   |

**Average Precision = sum of precisions at relevant ranks / total relevant docs**  
 $= (1.0 + 0.667 + 0.75 + 0.8 + 0.833 + 0.6 + 0.636 + 0.571) / 8$   
 $= 5.857 / 8$   
 $= 73.21\%$

#### Key things to remember:

- **P@k** = precision at a specific rank position
- **Average Precision** = only count precision at ranks where relevant docs appear, divide by total relevant docs
- Don't include non-relevant ranks in the sum!

### Q4 - MAP & Standard Recall Levels: Key Concepts (from the PDF)

#### Two new concepts:

#### MAP (Mean Average Precision)

- Average of the **Average Precision** scores across multiple queries
- Formula:

$$MAP = \frac{\sum_q AveragePrecision(q)}{number\ of\ queries}$$

#### From the PDF:

- Average Precision Query 1 =  $(1.0 + 0.67 + 0.5 + 0.44 + 0.5) / 5 = 0.62$
- Average Precision Query 2 =  $(0.5 + 0.4 + 0.43) / 3 = 0.44$
- **MAP** =  $(0.62 + 0.44) / 2 = 0.53$

#### Standard Recall Levels

- Recall levels: **0, 0.1, 0.2, 0.3, ... 1.0** (11 levels)
- Precision at recall level R:

$$P(R) = \max\{P' : R' \geq R \wedge (R', P') \in S\}$$

**In plain English:** For recall level R, find all (R', P') pairs in S where  $R' \geq R$ , then take the **maximum precision** from those pairs.

#### Set S for Query 1 (from the PDF):

$S = \{(0.2, 1.0), (0.2, 0.5), (0.4, 0.67), (0.4, 0.5), (0.4, 0.4), (0.6, 0.5), (0.6, 0.43), (0.6, 0.38), (0.8, 0.44), (1.0, 0.5)\}$

#### Calculating P at each standard recall level:

| Recall Level | Qualifying pairs ( $R' \geq R$ )                               | Max Precision |
|--------------|----------------------------------------------------------------|---------------|
| 0, 0.1, 0.2  | All pairs                                                      | <b>1.0</b>    |
| 0.3, 0.4     | $R' \geq 0.3 \rightarrow \{0.67, 0.5, 0.4, 0.43, 0.38, 0.44\}$ | <b>0.67</b>   |
| 0.5, 0.6     | $R' \geq 0.5 \rightarrow \{0.5, 0.43, 0.38, 0.44\}$            | <b>0.5</b>    |
| 0.7, 0.8     | $R' \geq 0.7 \rightarrow \{0.44, 0.5\}$                        | <b>0.5</b>    |
| 0.9, 1.0     | $R' \geq 0.9 \rightarrow \{0.5\}$                              | <b>0.5</b>    |

#### Python list comprehension (from the PDF):

```
[max([p for (r,p) in S if r >= i/10]) for i in range(11)]
```

#### Breaking it down:

- `for i in range(11)` → loops through  $i = 0, 1, 2, \dots, 10$  (recall levels 0.0 to 1.0)
- `r >= i/10` → filters pairs where recall  $R' \geq$  current recall level
- `max(...)` → takes the maximum precision from qualifying pairs

#### Key things to remember:

- **MAP** = average of average precisions across all queries
- **P(R)** = maximum precision at or beyond recall level R
- Standard recall levels = **0 to 1.0 in steps of 0.1**
- Higher recall level → fewer qualifying pairs → precision generally decreases

### Q1 - Edit Distance & Stemming

- Stem class = group of words mapping to the same stem
- Edit distance = minimum number of operations to transform one word to another
- Damerau-Levenshtein = **minimum** (not maximum) insertions, deletions, substitutions, transpositions
- Speed up tricks: same first letter, same length, same sound

### Q2 - Noisy Channel Model

- $P(w|e) \propto P(e|w) \times P(w)$
- When edit distance and prior probability are equal → **context** (previous word) decides
- "fish tank" more common than "fish think" → "tank" wins

### Q3 - Precision at Rank k & Average Precision

- **P@k** = relevant docs in top-k / k
- Average Precision = sum of precisions at relevant ranks / total relevant docs
- Only count precision where relevant docs appear!

### Q4 - MAP & Standard Recall Levels

- MAP = average of Average Precisions across all queries
- **P(R)** = max precision where  $R' \geq R$  in set S
- Standard recall levels = 0 to 1.0 in steps of 0.1
- Python: `[max([p for (r,p) in S if r >= i/10]) for i in range(11)]`

## WEEK 7

| # | Topic | Type |
|---|-------|------|
|---|-------|------|

|    |                                            |                  |
|----|--------------------------------------------|------------------|
| Q1 | Mutual Information Measure (MIM)           | Concept + Python |
| Q2 | Term Frequency Measure                     | Calculation      |
| Q3 | Probabilistic Information Filtering - BM25 | Concept          |
| Q4 | LLMs & RAG - TRUE/FALSE                    | Concept          |

## Q1 - Mutual Information Measure (MIM): Key Concepts (from the PDF)

The formula:

$$MIM = \log \frac{P(a, b)}{P(a) \times P(b)}$$

Where:

- **P(a)** = probability that word a occurs in a text window
- **P(b)** = probability that word b occurs in a text window
- **P(a,b)** = probability that both a and b occur in the **same** text window

### Part (1) - Expressing MIM using df functions:

Let  $N = |D|$  (total number of documents), then:

- $P(a) = df(D, a) / N$
- $P(b) = df(D, b) / N$
- $P(a,b) = df2(D, a, b) / N$

So:

$$MIM = \log \frac{df2(D, a, b) / N}{(df(D, a) / N) \times (df(D, b) / N)}$$

Simplified:

$$MIM = \log \frac{N \times df2(D, a, b)}{df(D, a) \times df(D, b)}$$

### Part (2) - Python function df2():

```
def df2(docs, a, b):
    """Calculate the number of documents that include
    both terms."""
    nab = 0
    for id, doc in docs.items():
        if (a in doc) and (b in doc):
            nab = nab + 1
    return nab
```

- Loops through all documents
- Counts documents where **both** term a AND term b appear

### Part (3) - Python function MIM():

```
import math
def MIM(docs, a, b):
    t_df = c_df(docs) # get df for all terms
    na = t_df[a] # df of term a
    nb = t_df[b] # df of term b
    nab = df2(docs, a, b) # docs containing both a
    and b
    _mim = math.log((len(docs)*nab/(na*nb)), 2)
    return _mim
```

Key things to remember:

- MIM measures how often two words **co-occur** compared to what would be expected by chance
- **High MIM** → words appear together more than expected → likely related

- **Low/negative MIM** → words appear together less than expected → unlikely related
- `df2()` counts documents with **both** terms
- `c_df()` returns df for **all** terms as a dictionary

## Q2 - Term Frequency Measure: Key Concepts (from the PDF)

The formula:

$$tf(t_k) = \frac{\sum_{d_i \in D^+} f_{ik}}{\sum_{d_i \in D^+} \sum_{t_j \in T} f_{ij}}$$

What each variable means:

| Variable              | Meaning                                                     |
|-----------------------|-------------------------------------------------------------|
| <b>D+</b>             | Set of <b>relevant</b> documents                            |
| <b>D-</b>             | Set of <b>non-relevant</b> documents                        |
| <b>f<sub>ik</sub></b> | Count of term t <sub>k</sub> in document d <sub>i</sub>     |
| <b>T</b>              | Full set of all terms                                       |
| <b>Numerator</b>      | Total count of term t <sub>k</sub> across all relevant docs |
| <b>Denominator</b>    | Total count of <b>all</b> terms across all relevant docs    |

In plain English:

- How many times does the term appear in relevant documents?
- Divided by the total number of words in all relevant documents

From the PDF:

- **D+ = {D2, D3, D4}** (relevant docs)
- **D- = {D1, D5, D6, D7}** (non-relevant docs)

Frequency table for relevant docs only:

| Doc | US | BILL | ECONOM | ESPIONAG | ... | Sum |
|-----|----|------|--------|----------|-----|-----|
| D2  | 2  | 0    | 1      | 0        | ... | 4   |
| D3  | 1  | 1    | 1      | 1        | ... | 4   |
| D4  | 1  | 1    | 1      | 1        | ... | 4   |

Calculating **tf(ESPIONAG)**:

- Numerator = ESPIONAG counts in D+ = 0 + 1 + 1 = **2**
- Denominator = total words in D+ = 4 + 4 + 4 = **12**
- **tf(ESPIONAG) = 2/12 = 0.167**

Key things to remember:

- Only use **relevant documents (D+)** in the calculation
- Numerator = term count across all relevant docs
- Denominator = **total word count** across all relevant docs (not just the term!)

## Q3 - Probabilistic Information Filtering (BM25): Key Concepts (from the PDF)

The BM25 weight formula:

$$w_5(t) = \log \frac{\frac{r(t)+0.5}{R-r(t)+0.5}}{\frac{n(t)-r(t)+0.5}{(N-n(t))-(R-r(t))+0.5}}$$

What each variable means:

| Variable | Meaning | Python definition |
|----------|---------|-------------------|
|----------|---------|-------------------|

|             |                                                       |                                                                                            |
|-------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>N</b>    | Total number of documents in training set D           | $N = \text{len}(D)$                                                                        |
| <b>R</b>    | Number of relevant documents                          | $R = \text{len}(\text{Rel})$                                                               |
| <b>n(t)</b> | Number of documents containing term t                 | $n(t) = \text{len}([d \text{ for } d \text{ in } D \text{ if } t \text{ in } d])$          |
| <b>r(t)</b> | Number of <b>relevant</b> documents containing term t | $r(t) = \text{len}([d \text{ for } d \text{ in } \text{Rel} \text{ if } t \text{ in } d])$ |

#### Example from the PDF:

$D = [['t1', 't2'], ['t3', 't2'], ['t1', 't2', 't5', 't4'], ['t3', 't6', 't4']]$   
 $\text{Rel} = [['t1', 't2', 't5', 't4'], ['t3', 't6', 't4']]$

$\text{len}(D)$  #  $N = 4$   
 $\text{len}(\text{Rel})$  #  $R = 2$   
 $\text{len}([d \text{ for } d \text{ in } D \text{ if } 't1' \text{ in } d])$  #  $n(t1) = 2$   
 $\text{len}([d \text{ for } d \text{ in } \text{Rel} \text{ if } 't1' \text{ in } d])$  #  $r(t1) = 1$

#### Key things to remember:

- **N** = all documents (relevant + non-relevant)
- **R** = relevant documents only
- **n(t)** = searches through **all** documents D
- **r(t)** = searches through **relevant** documents Rel only
- The formula adds **0.5** to avoid division by zero

#### Q4 - LLMs & RAG: Key Concepts (from the PDF)

##### Five statements - one is FALSE:

##### (1) LLMs and explainability ✓ TRUE

- LLMs can naturally generate **human-readable explanations** for recommendations
- This improves system **transparency and interpretability**

##### (2) LLM-based information filtering ✓ TRUE

- Does **not require labelled training data**
- Well-designed **prompts** can effectively guide the model's behaviour

##### (3) Retrieval-Augmented Generation (RAG) ✓ TRUE

- An AI framework that **enhances LLMs** by retrieving relevant information from **external knowledge bases**
- Leads to more **accurate and factually grounded** outputs

##### (4) LLMs in real-world news recommendation - THIS IS THE FALSE ONE

- Statement says: "effectiveness in real-world news recommendation systems has been **fully established**"
- This is **FALSE** - their effectiveness has **yet to be fully established**
- LLMs like GPT-4 have strong NLU capabilities but real-world news recommendation is still being researched

##### (5) Chain-of-Thought (CoT) prompting ✓ TRUE

- A method to improve LLMs by prompting them to generate **step-by-step reasoning** before giving a final answer

#### Key things to remember:

- **RAG** = retrieves from external knowledge bases → more accurate outputs
- **CoT** = step-by-step reasoning before final answer
- LLMs in news recommendation → **not yet fully established**

#### Q1 - Mutual Information Measure (MIM)

- $\text{MIM} = \log[P(a,b) / P(a) \times P(b)]$
- Simplified:  $\log[N \times \text{df2}(D,a,b) / \text{df}(D,a) \times \text{df}(D,b)]$
- High MIM → words co-occur more than expected → likely related
- $\text{df2}()$  counts docs with **both** terms
- $c\_df()$  returns df dictionary for all terms

#### Q2 - Term Frequency Measure

- $\text{tf}(tk) = \text{total count of term in } D+ / \text{total word count in } D+$
- Only use **relevant documents (D+)**
- $\text{tf}(\text{ESPIONAG}) = 2/12 = 0.167$

#### Q3 - BM25 Probabilistic Filtering

- $N = \text{len}(D) \rightarrow$  all documents
- $R = \text{len}(\text{Rel}) \rightarrow$  relevant documents only
- $n(t) =$  docs containing term t (all docs)
- $r(t) =$  relevant docs containing term t
- Adds 0.5 to avoid division by zero

#### Q4 - LLMs & RAG

- RAG = retrieves from external knowledge bases → factually grounded outputs
- CoT = step-by-step reasoning before final answer
- LLM-based filtering = no labelled training data needed
- LLMs in news recommendation = **not yet fully established** (FALSE statement)

### WEEK 8

| #  | Topic                                                   | Type    |
|----|---------------------------------------------------------|---------|
| Q1 | Query Likelihood, KL Divergence, Smoothing - TRUE/FALSE | Concept |
| Q2 | Pseudo Relevance Feedback - IF model design             | Concept |
| Q3 | CoT Prompt for Query Expansion                          | Concept |
| Q4 | Query Likelihood for Tweets - variable interpretation   | Concept |

#### Q1 - Query Likelihood, KL Divergence & Smoothing: Key Concepts (from the PDF)

##### Four statements to understand:

##### (1) Query Likelihood Retrieval Model ✓ TRUE

- Documents are ranked by the **probability that the query text could be generated** by the document language model
- i.e. rank by  $P(Q|D)$

##### (2) Unigram Language Model ✓ TRUE

- Used to estimate  **$P(Q|D)$**  - document D's score for query  $Q = \{q1, q2, \dots, qn\}$
- Needs individual **language model probabilities  $P(qi|D)$**  for each query term
- $i = 1, 2, \dots, n$

(3) Document Likelihood Model ✓ TRUE

- $P(D|Q)$  - probability of document given query
- More **theoretical** in information filtering context
- Because there is **no explicit query** available in filtering

(4) KL Divergence - THIS IS THE FALSE ONE

- Statement says: KL divergence is a technique for **avoiding estimation problems and overcoming data sparsity**
- This is **FALSE** - that describes **Smoothing**
- **KL divergence** is actually a **measure to estimate the difference between two probability distributions**
- **Smoothing** is the technique for avoiding data sparsity

Key things to remember:

- **KL divergence** = measures difference between two probability distributions
- **Smoothing** = technique to overcome data sparsity
- The two are **different things** - don't mix them up!

Q2 - Pseudo Relevance Feedback: Key Concepts (from the PDF)

The problem:

- User gives query Q and unlabelled collection C
- No labelled data available
- Need to build a probabilistic IF model using **pseudo relevance feedback**

Two phases from the PDF:

Phase 1 - Generate a training set D using Pseudo-Feedback Algorithm:

7 steps:

1. **Rank** documents in C in descending order using query Q
2. **Select top-k1** documents (e.g. k1=20) as **Rel1** - possible relevant documents
3. **Select important topical features T** from Rel1 (e.g. words with high  $tf \times idf$  weight)
4. **Calculate  $P(w|d)$**  for each word w in T using smoothing:

$$P(w|d) = (1 - \lambda)tf(w, d) + \lambda tf(w, C)$$
5. **Rank documents** in Rel1 using KL-divergence score
6. **Select top-k2** (e.g. k2=10) documents as **Rel2**
7. **Set:** D = C, D+ = Rel2, D- = C - Rel2

Phase 2 - Design a probabilistic IF model:

Based on two assumptions:

- **I2** - distribution of terms in relevant/non-relevant documents is **independent**
- **O1** - relevance is based only on the **presence** of search terms in documents

Use the **BM25 term weighting formula** (from Week 7):

- $N = \text{len}(D)$
- $R = \text{len}(D+)$
- $n(t) = \text{len}([d \text{ for } d \text{ in } D \text{ if } t \text{ in } d])$
- $r(t) = \text{len}([d \text{ for } d \text{ in } D+ \text{ if } t \text{ in } d])$

Key things to remember:

- Pseudo relevance feedback = **no labels needed** → uses top-k ranked docs as "assumed relevant"
- Two phases: **generate training set** → **build IF model**
- $k1 > k2$  (e.g. k1=20 for initial pool, k2=10 for final relevant set)
- Smoothing used to refine  $P(w|d)$

Q3 - CoT Prompt for Query Expansion: Key Concepts (from the PDF)

What is the task?

- Design a **Pseudo-Relevance Feedback style Chain-of-Thought (CoT) prompt** for query expansion
- Original query: **"climate change impact"**

The prompting template from the PDF:

Task: Expand the query using pseudo-relevance feedback.  
Query: [Original query]  
Top documents: [Short snippets]  
Instructions: Briefly explain your reasoning in 2-3 steps, then provide the expanded query.

The solution prompt from the PDF:

Assume the top retrieved documents are relevant. Think step by step to identify important terms appearing in these documents that can be used for query expansion. Remove overly general or non-informative terms.

Original query: "climate change impact"

Top documents (snippets):  
["Climate change impacts, risks and adaptation",  
"Global warming likely to reach 2°C before 2050...",  
"Marine sediments provide clues for future impact...",  
"What is climate change? A really simple guide",  
"Accounting for ocean impacts nearly doubles the social cost of carbon"]

Output: A ranked list of expansion terms and the expanded query.

Expanded query result:

"climate change impact global warming climate impacts climate risks adaptation ocean impacts social cost of carbon marine ecosystems temperature increase"

Key things to remember:

| Concept                   | Meaning                                                     |
|---------------------------|-------------------------------------------------------------|
| Query expansion           | Adding new terms to original query to improve retrieval     |
| Pseudo relevance feedback | Assume top-k results are relevant → extract terms from them |
| CoT prompting             | Ask LLM to reason <b>step by step</b> before giving answer  |
| "Think step by step"      | Key phrase that triggers CoT reasoning in LLMs              |
| Remove general terms      | Filter out non-informative words from expansion             |

## Q4 - Query Likelihood for Tweets: Key Concepts (from the PDF)

The formula:

$$f(d, Q) = \sum_{w \in Q \cap d} c(w, Q) \log \left( 1 + \frac{c(w, d)}{\mu \times P(w|C)} \right) + |Q| \log \frac{\mu}{\mu + |d|}$$

What each variable means:

| Variable   | Meaning                                                                      |
|------------|------------------------------------------------------------------------------|
| $c(w, d)$  | Count of word $w$ in the given tweet $d$                                     |
| $c(w, Q)$  | Count of word $w$ in the given query $Q$                                     |
| $P(w C)$   | Probability of word $w$ in the collection $C$ - used for model normalisation |
| $ d $      | Length (size) of tweet $d$                                                   |
| $ Q $      | Length (size) of query $Q$                                                   |
| $\mu$ (mu) | Smoothing parameter                                                          |

Key things to remember:

- $c(w, d)$  → looks at the **tweet**
- $c(w, Q)$  → looks at the **query**
- $P(w|C)$  → looks at the **entire collection** - normalises the model
- The sum is only over words that appear in **both** the query AND the tweet ( $w \in Q \cap d$ )
- $\mu$  controls the smoothing - balances between document probability and collection probability

In plain English:

- For each word that appears in **both** the query and the tweet → calculate a score
- Words that appear **more in the tweet** relative to the collection → higher score
- $\mu$  prevents division by zero and handles smoothing

### Q1 - Query Likelihood, KL Divergence & Smoothing

- Query likelihood = rank documents by  $P(Q|D)$
- Unigram model = estimate  $P(q_i|D)$  for each query term
- Document likelihood  $P(D|Q)$  = more theoretical, no explicit query in filtering
- KL divergence** = measures difference between two probability distributions
- Smoothing** = technique to overcome data sparsity (NOT KL divergence!)

### Q2 - Pseudo Relevance Feedback

- Phase 1: rank → select top-k1 → extract features → rank again → select top-k2 → set  $D^+$
- Phase 2: use BM25 with  $N, R, n(t), r(t)$
- No labels needed → assume top-k docs are relevant
- Smoothing:  $P(w|d) = (1-\lambda)\text{tf}(w, d) + \lambda\text{tf}(w, C)$

### Q3 - CoT Prompt for Query Expansion

- CoT = "think step by step" before giving answer
- Pseudo relevance feedback → assume top docs relevant → extract expansion terms
- Remove overly general/non-informative terms
- Expanded query adds more specific related terms

### Q4 - Query Likelihood for Tweets

- $c(w, d)$  = count of word  $w$  in tweet
- $c(w, Q)$  = count of word  $w$  in query

- $P(w|C)$  = probability of word in collection → normalisation
- $\mu$  = smoothing parameter
- Sum only over words in both query AND tweet ( $Q \cap d$ )

## WEEK 9

| #  | Topic                                                                       | Type             |
|----|-----------------------------------------------------------------------------|------------------|
| Q1 | Multinomial Document Representation & Naïve Bayes - $P(c)$ and $P(w_i c_j)$ | Calculation      |
| Q2 | Naïve Bayes Classification - Python implementation                          | Python coding    |
| Q3 | Rocchio Classification - centroid calculation                               | Python coding    |
| Q4 | Sentiment Analysis - TRUE/FALSE                                             | Concept          |
| Q5 | Sentiment Analysis - NLTK aspects & ABSA prompt                             | Python + Concept |

## Q1 - Multinomial Document Representation & Naïve Bayes: Key Concepts (from the PDF)

The training set (Table 1):

| Doc | cheap | buy | banking | dinner | the | class    |
|-----|-------|-----|---------|--------|-----|----------|
| 1   | 0     | 0   | 0       | 0      | 2   | not spam |
| 2   | 3     | 0   | 1       | 0      | 1   | spam     |
| 3   | 0     | 0   | 0       | 0      | 1   | not spam |
| 4   | 2     | 0   | 3       | 0      | 2   | spam     |
| 5   | 5     | 2   | 0       | 0      | 1   | spam     |
| 6   | 0     | 0   | 1       | 0      | 1   | not spam |
| 7   | 0     | 1   | 1       | 0      | 1   | not spam |
| 8   | 0     | 0   | 0       | 0      | 1   | not spam |
| 9   | 0     | 0   | 0       | 0      | 1   | not spam |
| 10  | 1     | 1   | 0       | 1      | 2   | not spam |

- $C = \{c1, c2\} = \{\text{'spam'}, \text{'not spam'}\}$
- $V = \{\text{'cheap'}, \text{'buy'}, \text{'banking'}, \text{'dinner'}, \text{'the'}\} \rightarrow |V| = 5$

Part (1) - Calculate  $P(c)$  for all  $c \in C$ :

- $C1$  (spam) = {d2, d4, d5} →  $N_{c1} = 3$
- $C2$  (not spam) = {d1, d3, d6, d7, d8, d9, d10} →  $N_{c2} = 7$
- $N = |D| = 10$

$$P(c1) = \frac{N_{c1}}{N} = \frac{3}{10} = 0.3$$

$$P(c2) = \frac{N_{c2}}{N} = \frac{7}{10} = 0.7$$



Part (2) - Calculate  $P(w_i|c_j)$  using Laplacian smoothing:

$$P(w_i|c_j) = \frac{tf_{w_i,c_j} + 1}{|c_j| + |V|}$$

Where:

- $tf_{w_i,c_j}$  = total count of term  $w_i$  in all documents of class  $c_j$
- $|c_j|$  = total number of ALL terms in class  $c_j$
- $|V|$  = vocabulary size = 5

First calculate total counts for spam (c1) - docs D2, D4, D5:

| Term    | D2 | D4 | D5 | Total (tfw,c1) |
|---------|----|----|----|----------------|
| cheap   | 3  | 2  | 5  | 10             |
| buy     | 0  | 0  | 2  | 2              |
| banking | 1  | 3  | 0  | 4              |
| dinner  | 0  | 0  | 0  | 0              |
| the     | 1  | 2  | 1  | 4              |
| **      | c1 | ** |    |                |

For not spam (c2) - docs D1, D3, D6, D7, D8, D9, D10:

| Term    | Total (tfw,c2) |
|---------|----------------|
| cheap   | 1              |
| buy     | 2              |
| banking | 2              |
| dinner  | 1              |
| the     | 9              |
| **      | c2             |

Final  $P(w_i|c_j)$  table:

| Term    | c1 (spam)                      | c2 (not spam)               |
|---------|--------------------------------|-----------------------------|
| cheap   | $(10+1)/(20+5) = 11/25 = 0.44$ | $(1+1)/(15+5) = 2/20 = 0.1$ |
| buy     | $(2+1)/25 = 3/25 = 0.12$       | $(2+1)/20 = 3/20 = 0.15$    |
| banking | $(4+1)/25 = 5/25 = 0.2$        | $(2+1)/20 = 3/20 = 0.15$    |
| dinner  | $(0+1)/25 = 1/25 = 0.04$       | $(1+1)/20 = 2/20 = 0.1$     |
| the     | $(4+1)/25 = 5/25 = 0.2$        | $(9+1)/20 = 10/20 = 0.5$    |

Key things to remember:

- $P(c)$  = class size / total documents
- $P(w_i|c_j)$  uses **Laplacian smoothing** → add 1 to numerator, add  $|V|$  to denominator
- $|c_j|$  = total word count in that class (NOT number of documents!)

- Smoothing ensures no term gets **zero probability**

## Q2 - Naïve Bayes Classification: Key Concepts (from the PDF)

The key formula:

$$P(c|d) = \log(P(c)) + \sum_{w_i \in d} \log(P(w_i|c))$$

Two functions to understand:

TRAIN\_MULTINOMIAL\_NB(C, D, V)

**Purpose:** Calculate  $P(c)$  and  $P(w|c)$  for all terms and classes

```
def n_words(c, C, D):
    """Count total words in documents labelled as
    class c"""
    n_w = 0
    for i in range(len(C)):
        if C[i] == c:
            for j in range(len(D[i])):
                n_w = n_w + D[i][j]
    return n_w

def TRAIN_MULTINOMIAL_NB(C, D, V):
    N = len(D)
    prior = []
    condprob = tf = [[0 for w in range(len(V))] for c
    in range(2)]
    for c in range(2):
        Nc = 0
        for i in C:
            if C[i] == c:
                Nc = Nc + 1
        prior.append(Nc/len(C))
        for w in range(len(V)):
            # calculate
            tf(c,w)
            for d in range(len(D)):
                if C[d] == c:
                    tf[c][w] = tf[c][w] + D[d][w]
            for w in range(len(V)):
                condprob[c][w] =
                (tf[c][w]+1)/(n_words(c,C,D)+len(V))
    return(prior, condprob)
```

**Output:**

```
prior = [0.7, 0.3] # P(c0), P(c1)
condprob = [[0.1, 0.15, 0.15, 0.1, 0.5], # c0 (not
spam)
            [0.44, 0.12, 0.2, 0.04, 0.2]] # c1
(spam)
```

APPLY\_MULTINOMIAL\_NB(V, prior, condprob, d)

**Purpose:** Assign class label to a new document d

```
import math
def APPLY_MULTINOMIAL_NB(V, prior, condprob, d):
    W = {V[i]:i for i in range(len(V)) if V[i] in d}
    score = {}
    for c in range(2):
        score[c] = math.log(prior[c])
        for (w, i) in W.items():
            score[c] = score[c] +
            math.log(condprob[c][i])
    if score[1] > score[0]:
        return 1 # spam
    else:
        return 0 # not spam
```

### Key steps:

1. Find which vocabulary terms appear in document  $d \rightarrow W$
2. For each class, calculate log score =  $\log(P(c)) + \text{sum of } \log(P(w|c))$
3. Return class with **higher score**

### Key things to remember:

- Use **log** to avoid underflow (multiplying many small probabilities)
- Only consider terms that appear in **both V and d**
- Higher score  $\rightarrow$  assigned class
- $\text{prior}[0] = P(\text{not spam}), \text{prior}[1] = P(\text{spam})$

## Q3 - Rocchio Classification: Key Concepts (from the PDF)

### What is Rocchio Classification?

- Uses **centroids** to define boundaries between classes
- Centroid = **vector average** of all documents in a class
- A new document is assigned to the class whose centroid is **most similar** to it

### The training data (from the PDF):

| Doc | V   | US  | Sp  | Sal | Ma  | G   | Espion | Econ | Chi  | Bil | Class |
|-----|-----|-----|-----|-----|-----|-----|--------|------|------|-----|-------|
| 394 | 0.1 | 0.0 | 0.2 | 0.0 | 0.0 | 0.2 | 0.12   | 0.11 | 0.00 | 0.0 | 1     |
| 96  | 7   | 1   | 0   | 0   | 2   | 0   |        |      |      | 0   |       |
| 465 | 0.1 | 0.2 | 0.1 | 0.0 | 0.0 | 0.0 | 0.22   | 0.20 | 0.00 | 0.1 | 1     |
| 47  | 0   | 1   | 0   | 0   | 0   | 0   |        |      |      | 0   |       |
| 469 | 0.0 | 0.2 | 0.1 | 0.2 | 0.0 | 0.1 | 0.10   | 0.10 | 0.01 | 0.0 | 1     |
| 74  | 0   | 3   | 0   | 0   | 5   | 0   |        |      |      | 0   |       |
| 623 | 0.1 | 0.0 | 0.2 | 0.0 | 0.0 | 0.2 | 0.12   | 0.11 | 0.00 | 0.0 | 1     |
| 25  | 7   | 1   | 0   | 0   | 2   | 0   |        |      |      | 0   |       |
| 614 | 0.1 | 0.0 | 0.0 | 0.3 | 0.1 | 0.2 | 0.00   | 0.12 | 0.10 | 0.0 | 0     |
| 6   | 0   | 0   | 0   | 0   | 0   | 0   |        |      |      | 0   |       |
| 185 | 0.0 | 0.3 | 0.0 | 0.3 | 0.2 | 0.0 | 0.00   | 0.20 | 0.15 | 0.2 | 0     |
| 86  | 0   | 0   | 0   | 0   | 0   | 0   |        |      |      | 0   |       |
| 221 | 0.2 | 0.0 | 0.0 | 0.1 | 0.2 | 0.2 | 0.00   | 0.00 | 0.00 | 0.1 | 0     |
| 70  | 0   | 0   | 0   | 5   | 0   | 5   |        |      |      | 0   |       |

- **Relevant (class=1):** 39496, 46547, 46974, 62325  $\rightarrow$  4 docs
- **Non-relevant (class=0):** 6146, 18586, 22170  $\rightarrow$  3 docs

### How to calculate centroid:

$$\text{centroid} = \frac{\sum_{d \in \text{class}} \text{vector}(d)}{|\text{class}|}$$

### Example - VM for relevant centroid (R101):

- $(0.17 + 0.10 + 0.00 + 0.17) / 4 = 0.44/4 = 0.11$

### Example - VM for non-relevant centroid (!R101):

- $(0.10 + 0.00 + 0.20) / 3 = 0.30/3 = 0.10$

### Final centroids from the PDF:

#### R101 (relevant):

```
{'VM': 0.11, 'US': 0.115, 'Spy': 0.15, 'Sale': 0.05, 'Man': 0.0225, 'GM': 0.125, 'Espionag': 0.14, 'Econom': 0.13, 'Chief': 0.0025, 'Bill': 0.025}
```

#### !R101 (non-relevant):

```
{'VM': 0.10, 'US': 0.10, 'Spy': 0.0, 'Sale': 0.25,
```

```
'Man': 0.167, 'GM': 0.15, 'Espionag': 0.0, 'Econom': 0.107, 'Chief': 0.083, 'Bill': 0.10}
```

### The Python function logic:

```
def train_Rocchio(train_set, topic_set, pos_cls):  
    # 1. Loop through all documents  
    # 2. If relevant  $\rightarrow$  add to positive centroid sum,  
    increment pos_n  
    # 3. If not relevant  $\rightarrow$  add to negative centroid  
    sum, increment neg_n  
    # 4. Divide each centroid by its count  
    # 5. Return both centroids
```

### Key things to remember:

- Centroid = **average vector** of all docs in a class
- Relevant centroid named **R101**, non-relevant named **!R101**
- Uses `try/except KeyError` to build centroid dict (same pattern as Week 2!)
- Divide by **pos\_n** for relevant, **neg\_n** for non-relevant

## Q4 - Sentiment Analysis: Key Concepts (from the PDF)

### Five statements - one is FALSE:

#### (1) Sentiment Analysis $\checkmark$ TRUE

- Also called **opinion mining**
- Discovers users' opinions about **products or services** in online reviews
- Also observes trends in **public mood** and analyses **clinical records**

#### (2) Orientation $\checkmark$ TRUE

- The opinion provided about the **entity and/or aspect**
- Provided by the **opinion holder** at a **specific time**

#### (3) Emotion Classification - THIS IS THE FALSE ONE

- Statement says: "goal of emotion classification is to **separate subjective from objective** information - a binary classification task"
- This is **FALSE** - that describes **subjectivity classification**
- **Emotion classification** = classify text according to a **predefined set of basic emotions** (e.g. happy, sad, angry)
- **Subjectivity classification** = separate subjective from objective information (binary task)

#### (4) Aspects $\checkmark$ TRUE

- Aspects are **features, components or functions** of the entity
- They can be **nouns and/or noun phrases**

#### (5) Polarity Classification $\checkmark$ TRUE

- Groups expressed opinion into **positive, negative or neutral** regions
- Can be applied to a document, sentence, or entity feature/aspect

### Key things to remember:

| Term                               | Meaning                                            |
|------------------------------------|----------------------------------------------------|
| <b>Sentiment analysis</b>          | Discover opinions about products/services          |
| <b>Orientation</b>                 | Opinion about entity/aspect by opinion holder      |
| <b>Emotion classification</b>      | Classify by predefined basic emotions              |
| <b>Subjectivity classification</b> | Separate subjective vs objective (binary)          |
| <b>Aspects</b>                     | Features/components of entity - nouns/noun phrases |
| <b>Polarity classification</b>     | Positive / negative / neutral grouping             |

## Q5 - Sentiment Analysis: NLTK & ABSA Prompt

### The sentence from the PDF:

"The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outstanding; however, the battery life is disappointing."

### Part (a) - get\_aspect() using NLTK:

**Key idea:** Extract all tokens whose POS tag starts with "NN" (nouns)

```
from nltk import word_tokenize, pos_tag
```

```
def get_aspect(sentence):
    tokens = word_tokenize(sentence)
    tagged = pos_tag(tokens)
    aspects = [word for word, tag in tagged if
tag.startswith("NN")]
    return aspects
```

### Output:

```
['phone', 'quality', 'performs', 'games', 'camera',
'quality', 'battery', 'life']
```

### POS tags starting with "NN":

- NN = singular noun
- NNS = plural noun
- NNP = proper noun singular
- NNPS = proper noun plural

### Part (b) - ABSA Prompt Design:

Please perform Unified Aspect-Based Sentiment Analysis task.

Given the sentence, tag all aspects. Aspect should be substring of the sentence. If there are no aspects, return an empty list; otherwise return a python list of aspects (tokens) in single quotes. Please return python list only, without any other comments or texts.

Sentence: We discuss their opinions about the new stadium.  
Tokens: ['stadium']

Sentence: The new stadium is beautiful, and the design is inspired by the Queenslander architecture style.  
Tokens: ['stadium', 'design']

Sentence: The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outstanding; however, the battery life is disappointing.  
Tokens:

### Part (c) - Comparison:

|                   | Method a) NLTK                                  | Method b) ABSA Prompt                                   |
|-------------------|-------------------------------------------------|---------------------------------------------------------|
| <b>Strengths</b>  | Explicit, interpretable, rule-based             | More accurate, semantically meaningful                  |
| <b>Weaknesses</b> | Lower precision, limited semantic understanding | Lacks transparency - unclear how aspects are identified |
| <b>Approach</b>   | POS tagging (NN tags)                           | LLM-based reasoning                                     |

### Key conclusion from the PDF:

- Method a) = **greater interpretability** but **weaker performance**
- Method b) = **better results** but **less explainability**

### Key things to remember:

- word\_tokenize() → splits sentence into tokens
- pos\_tag() → assigns POS tag to each token
- tag.startswith("NN") → filters nouns only
- ABSA prompt uses **few-shot examples** to guide the LLM
- **"Please return python list only"** → forces structured output

### Q1 - Multinomial Naïve Bayes

- $P(c) = N_c / N$  → class size / total documents
- $P(w_i|c) = (tf_{w_i,c} + 1) / (|c| + |V|)$  → Laplacian smoothing
- $|c|$  = total word count in class (not number of docs!)
- Smoothing adds 1 to numerator,  $|V|$  to denominator → no zero probabilities

### Q2 - Naïve Bayes Classification (Python)

- $P(c|d) = \log(P(c)) + \text{sum of } \log(P(w_i|c))$
- Use **log** to avoid underflow
- TRAIN → calculates prior and condprob
- APPLY → computes score for each class → returns highest

### Q3 - Rocchio Classification

- Centroid = vector average of all docs in a class
- $R_{101}$  = relevant centroid,  $\neg R_{101}$  = non-relevant centroid
- New doc assigned to class with **most similar centroid**
- Use try/except KeyError to build centroid dict

### Q4 - Sentiment Analysis (TRUE/FALSE)

- Emotion classification = classify by **predefined basic emotions**
- Subjectivity classification = **subjective vs objective** (binary) ← FALSE statement mixed these up
- Aspects = nouns/noun phrases (features of entity)
- Polarity = positive / negative / neutral

### Q5 - NLTK & ABSA

- word\_tokenize() → tokens, pos\_tag() → POS tags
- tag.startswith("NN") → extracts nouns as aspects
- ABSA prompt uses few-shot examples → forces structured output
- NLTK = interpretable but less accurate
- ABSA = more accurate but less explainable

## WEEK 11

| #  | Topic                                          | Type          |
|----|------------------------------------------------|---------------|
| Q1 | Distance Functions - Jaccard & Simple Matching | Calculation   |
| Q2 | Cluster Precision - MaxClass                   | Calculation   |
| Q3 | Recommender Systems - correlation & clustering | Python coding |

### Q1 - Distance Functions: Key Concepts (from the PDF)

The binary information table:

| Doc | t1 | t2 | t3 | t4 | t5 | t6 | t7 |
|-----|----|----|----|----|----|----|----|
| d1  | 1  | 1  | 0  | 0  | 0  | 0  | 0  |
| d2  | 0  | 0  | 1  | 1  | 0  | 1  | 0  |

#### Part (1) - Binary Contingency Table

Compare d1 and d2 term by term:

| Term | d1 | d2 | Category       |
|------|----|----|----------------|
| t1   | 1  | 0  | s (d1=1, d2=0) |
| t2   | 1  | 0  | s (d1=1, d2=0) |
| t3   | 0  | 1  | r (d1=0, d2=1) |
| t4   | 0  | 1  | r (d1=0, d2=1) |
| t5   | 0  | 0  | t (d1=0, d2=0) |
| t6   | 0  | 1  | r (d1=0, d2=1) |
| t7   | 0  | 0  | t (d1=0, d2=0) |

The 4 categories:

|      | d2=1 | d2=0 | Sum |
|------|------|------|-----|
| d1=1 | q=0  | s=2  | 2   |
| d1=0 | r=3  | t=2  | 5   |
| Sum  | 3    | 4    | 7   |

Where:

- **q** = both 1 (both have the term)
- **r** = d1=0, d2=1 (only d2 has it)
- **s** = d1=1, d2=0 (only d1 has it)
- **t** = both 0 (neither has the term)

#### Part (2) - Jaccard Distance:

$$disJac(d1, d2) = \frac{r + s}{q + r + s} = \frac{3 + 2}{0 + 3 + 2} = \frac{5}{5} = 1$$

#### Part (3) - Simple Matching Coefficient Distance:

$$disS(d1, d2) = \frac{r + s}{q + r + s + t} = \frac{3 + 2}{0 + 3 + 2 + 2} = \frac{5}{7}$$

Key differences between the two:

|             | Jaccard                   | Simple Matching             |
|-------------|---------------------------|-----------------------------|
| Formula     | $(r+s)/(q+r+s)$           | $(r+s)/(q+r+s+t)$           |
| Includes t? | No - ignores double zeros | Yes - includes double zeros |
| Best for    | Sparse binary data        | Dense binary data           |

Key things to remember:

- **q** = both 1, **t** = both 0 → agreements

- **r** and **s** = disagreements
- Jaccard **ignores t** (double zeros) → stricter measure
- Simple matching **includes t** → generally lower distance
- Higher distance = **less similar**

### Q2 - Cluster Precision: Key Concepts (from the PDF)

The document collection (same as Week 9 Q1):

| Doc | Class    |
|-----|----------|
| d1  | not spam |
| d2  | spam     |
| d3  | not spam |
| d4  | spam     |
| d5  | spam     |
| d6  | not spam |
| d7  | not spam |
| d8  | not spam |
| d9  | not spam |
| d10 | not spam |

True classes:

- **C\_spam** = {d2, d4, d5}
- **C\_not\_spam** = {d1, d3, d6, d7, d8, d9, d10}

Clusters produced by algorithm:

- **C1** = {d2, d5, d10}
- **C2** = {d1, d3, d4, d6, d7, d8, d9}

#### Part (1) - MaxClass(Ci):

**MaxClass** = the subset of documents in cluster Ci that belong to the **majority class** of that cluster

For C1 = {d2, d5, d10}:

- d2 = spam ✓
- d5 = spam ✓
- d10 = not spam ✗
- Majority class = **spam** (2 out of 3)
- **label(C1) = 'spam'**
- **MaxClass(C1) = {d2, d5}** → 2 docs

For C2 = {d1, d3, d4, d6, d7, d8, d9}:

- d1 = not spam ✓
- d3 = not spam ✓
- d4 = spam ✗
- d6 = not spam ✓
- d7 = not spam ✓
- d8 = not spam ✓
- d9 = not spam ✓
- Majority class = **not spam** (6 out of 7)
- **label(C2) = 'not\_spam'**
- **MaxClass(C2) = {d1, d3, d6, d7, d8, d9}** → 6 docs

#### Part (2) - Cluster Precision:

$$Cluster\ Precision = \frac{\sum_i |MaxClass(C_i)|}{|D|}$$

$$= \frac{|MaxClass(C1)| + |MaxClass(C2)|}{|D|} = \frac{2 + 6}{10} = 0.8$$

### Key things to remember:

- **MaxClass(Ci)** = docs in cluster Ci that match the **majority class label**
- **label(Ci)** = the majority class of cluster Ci
- **Cluster Precision** = sum of all MaxClass sizes / total documents
- Higher cluster precision = **better clustering**

### Q3 - Recommender Systems: Key Concepts (from the PDF)

#### What is the task?

- Build a recommender system using **user-based collaborative filtering**
- Uses **correlation** between users to find similar users
- Predicts ratings for **unseen items** based on similar users' ratings

#### Data representation:

```
I = [0, 1, 2, 3]          # 4 items numbered 0-3
U = [0, 1, 2, 3, 4]       # 5 users numbered 0-4
Rvs = [[1,2,1,0],         # user 0's ratings
        [0,1,1,1],        # user 1's ratings
        [2,1,3,5],        # user 2's ratings
        [1,0,2,0],        # user 3's ratings
        [0,2,3,4]]        # user 4's ratings
# 0 means the user has NOT yet rated the item
```

### Three functions from the PDF:

#### 1. my\_correlation(Rvs)

```
def my_correlation(Rvs):
    U = [i for i in range(len(Rvs))]
    I = [i for i in range(len(Rvs[0]))]
    Uc = [[1 for i in range(len(U))] for i in range(len(I))]
    ...
    return(U, I, Uc)
```

- Returns **U** (users), **I** (items), **Uc** (correlation matrix)
- Uc is a  $|U| \times |U|$  matrix of user-user similarities

#### 2. my\_cluster(Rvs)

```
def my_cluster(Rvs):
    ...
    return C # clusters - list of user lists
```

- Groups **similar users** into clusters based on correlation

#### 3. my\_prediction(Rvs, C, U, I)

```
def my_prediction(Rvs, C, U, I):
    ...
    return Rvs # updated Rvs with predicted ratings
```

- Fills in **0 ratings** with predicted values based on cluster members' ratings

### Key things to remember:

| Concept            | Meaning                                        |
|--------------------|------------------------------------------------|
| <b>Rvs</b>         | Rating vectors - rows = users, columns = items |
| <b>0 rating</b>    | User has <b>not yet rated</b> the item         |
| <b>Correlation</b> | Similarity measure between users               |
| <b>Cluster</b>     | Group of similar users                         |
| <b>Prediction</b>  | Fill in 0 ratings using similar users' ratings |
| <b>Uc</b>          | User-user correlation matrix (                 |

### The overall pipeline:

1. `my_correlation(Rvs)` → compute user similarities

2. `my_cluster(Rvs)` → group similar users
3. `my_prediction(Rvs, C, U, I)` → predict unseen ratings

### Q1 - Distance Functions

- **q** = both 1, **r** = d1=0,d2=1, **s** = d1=1,d2=0, **t** = both 0
- **Jaccard** =  $(r+s)/(q+r+s)$  → ignores double zeros (t)
- **Simple Matching** =  $(r+s)/(q+r+s+t)$  → includes double zeros
- Higher distance = less similar

### Q2 - Cluster Precision

- **MaxClass(Ci)** = docs in cluster Ci matching majority class label
- **label(Ci)** = majority class of that cluster
- **Cluster Precision** = sum of MaxClass sizes / total documents
- Example:  $(2+6)/10 = 0.8$

### Q3 - Recommender Systems

- **Rvs** = rating vectors (rows=users, columns=items)
- **0** = not yet rated
- Pipeline: correlation → cluster → prediction
- `my_correlation()` → user-user similarity matrix
- `my_cluster()` → group similar users
- `my_prediction()` → fill in unseen ratings

## WEEK 12

| #  | Topic                                            | Type          |
|----|--------------------------------------------------|---------------|
| Q1 | PageRank - calculate PR values                   | Calculation   |
| Q2 | Attention to BERT - attention mask               | Python coding |
| Q3 | Contrastive Learning + Pseudo Relevance Feedback | Concept       |

### Q1 - PageRank: Key Concepts (from the PDF)

#### The formula:

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

#### Where:

| Variable                             | Meaning                                     |
|--------------------------------------|---------------------------------------------|
| <b>N</b>                             | Total number of pages                       |
| <b><math>\lambda</math> (lambda)</b> | Damping factor = 0.15                       |
| <b>Bu</b>                            | Set of pages that <b>point to</b> page u    |
| <b>Lv</b>                            | Number of <b>outgoing links</b> from page v |
| <b>PR(v)</b>                         | PageRank of page v                          |

### The Web graph from the PDF:

- Pages:  $P = \{A, B, C, D\}$
- Links:  $L = \{(A \rightarrow B), (A \rightarrow D), (C \rightarrow A), (D \rightarrow B), (C \rightarrow B)\}$

#### Visualised:

```
C → A → B
C → B
A → D → B
```

#### Step 1 - Count outgoing links (Lv):

- **LC** =  $\{A, B\} = 2$  (C points to A and B)
- **LA** =  $\{B, D\} = 2$  (A points to B and D)
- **LD** =  $\{B\} = 1$  (D points to B)
- **LB** =  $\{\} = 0$  (B points to nothing)

#### Step 2 - Find Bu (pages pointing TO each page):

- **BA** =  $\{C\}$  → only C points to A



- **BB** = {A, C, D} → A, C and D all point to B
- **BC** = {} → nobody points to C
- **BD** = {A} → only A points to D

**Step 3 - Calculate PR for each page ( $\lambda=0.15$ ,  $N=4$ , initial  $PR=0.25$ ):**

**PR(A):**

$$= \frac{0.15}{4} + 0.85 \times \frac{PR(C)}{L_C} = 0.0375 + 0.85 \times \frac{0.25}{2} = \mathbf{0.14375}$$

**PR(B):**

$$= \frac{0.15}{4} + 0.85 \times \left( \frac{PR(C)}{2} + \frac{PR(A)}{2} + \frac{PR(D)}{1} \right)$$

$$= 0.0375 + 0.85 \times (0.125 + 0.125 + 0.25) = \mathbf{0.4625}$$

**PR(C)**

$$= \frac{0.15}{4} + 0.85 \times 0 = \mathbf{0.0375}$$

(No pages point to C → second term = 0)

**PR(D):**

$$= \frac{0.15}{4} + 0.85 \times \frac{PR(A)}{2} = 0.0375 + 0.85 \times \frac{0.25}{2} = \mathbf{0.14375}$$

**Final PR values after 1st iteration:**

| Page | PR value                 |
|------|--------------------------|
| A    | 0.14375                  |
| B    | <b>0.4625</b> (highest!) |
| C    | 0.0375 (lowest)          |
| D    | 0.14375                  |

**Key things to remember:**

- $\lambda/N$  = random jump probability → every page gets a small base score
- $(1-\lambda) \times \text{sum}$  = link contribution → pages with more inlinks get higher PR
- **B points to nothing** → only gets the base score  $\lambda/N$
- **C points to nothing incoming** → only gets base score
- **B has most inlinks** → highest PR

**Q2 - Attention to BERT: Key Concepts (from the PDF)**

**What is the task?**

- Extend Week 3's BERT code to understand how **attention\_mask** is used
- Compare **initial embeddings** vs **contextualised embeddings** after passing through encoder

**4 Steps from the PDF:**

**Step 1 - Tokenize input text:**

```
inputs = tokenizer(text1,
padding=True,
truncation=True,
return_tensors="pt",
add_special_tokens=True)
```

**Step 2 - Generate initial embeddings:**

```
embedding_layer = model.embeddings
embeddings = embedding_layer(
```

```
input_ids=inputs["input_ids"],
token_type_ids=inputs["token_type_ids"]
)
print(embeddings.shape) # torch.Size([1, 10, 768])
print(embeddings[0, 0, :20]) # first token embedding
```

- Shape: **[1, 10, 768]** → 1 sentence, 10 tokens, 768 dimensions
- These are **initial embeddings** - no context yet

**Step 3 - Pass through BERT encoder with attention\_mask:**

```
encoder_outputs = model.encoder(
embeddings,
attention_mask = inputs["attention_mask"].bool()
)
```

**What is attention\_mask?**

- A **binary mask**: 1 = real token, 0 = padding token
- Tells the encoder which tokens to **attend to** and which to **ignore**
- Shorter sentences get padded → padding tokens should be ignored

**Step 4 - Print contextualised representation:**

```
last_hidden_state = encoder_outputs.last_hidden_state
print(last_hidden_state.shape) # torch.Size([1, 10, 768])
print(last_hidden_state[0, 0, :20]) # first token contextualised
```

**Key difference between Step 2 and Step 4:**

|                       | Initial Embedding             | Contextualised Embedding                        |
|-----------------------|-------------------------------|-------------------------------------------------|
| <b>What</b>           | Token's own embedding         | Token's embedding after seeing all other tokens |
| <b>Context aware?</b> | ✗ No                          | ✓ Yes                                           |
| <b>Shape</b>          | [1, 10, 768]                  | [1, 10, 768]                                    |
| <b>Values</b>         | Different from contextualised | Richer - influenced by surrounding tokens       |

**Key things to remember:**

- `model.embeddings` → gets initial token embeddings
- `model.encoder` → passes through transformer layers
- `attention_mask` → 1 = attend, 0 = ignore (padding)
- `last_hidden_state` → final contextualised word representations
- Same shape before and after encoder but **values are different**

**Q3 - Contrastive Learning + Pseudo Relevance**

**Feedback: Key Concepts (from the PDF)**

**The goal:**

- Integrate **pseudo relevance feedback** with **contrastive learning**
- Train an encoder that makes query vectors **closer to positive docs** and **further from negative docs**

**Key terms first:**

| Term                        | Meaning                                                                      |
|-----------------------------|------------------------------------------------------------------------------|
| <b>Positive sample</b>      | Document that is relevant to the query                                       |
| <b>Negative sample</b>      | Document that is NOT relevant to the query                                   |
| <b>Contrastive learning</b> | Train encoder to increase similarity with positives, decrease with negatives |

|                  |                                                     |
|------------------|-----------------------------------------------------|
| <b>sim(Q, D)</b> | Similarity between query vector and document vector |
|------------------|-----------------------------------------------------|

### The pipeline from the PDF (3 steps):

#### Step 1 - Build Training Pairs:

- Use a **baseline IR model** (e.g. BM25) to retrieve top-k documents
- **Assume top-k docs = positive samples**
- Build positive pairs: **(Q, D+)** where  $D+ \in$  top-k docs

#### Step 2 - Select Negative Pairs:

- **Random documents** OR **low-ranked documents**
- **Hard negatives** = documents ranked just below top-k (most challenging)
- Build negative pairs: **(Q, D-)** where  $D- \notin$  top-k docs

#### Step 3 - Train Contrastive Model:

- Train encoder using **contrastive loss** (e.g. InfoNCE)
- Encoder learns to:
  - **Increase**  $\text{sim}(Q, D+)$  → query closer to positive docs
  - **Decrease**  $\text{sim}(Q, D-)$  → query further from negative docs

### Concrete example from the PDF:

Query Q: "deep learning for NLP"

#### Top-3 retrieved (positives):

- Doc1: "transformer models" ✓
- Doc2: "neural networks NLP" ✓
- Doc3: "deep learning overview" ✓

#### Negatives:

- Doc10: "cooking recipes" ✗
- Doc20: "sports news" ✗

#### Training pairs:

- **(Q, Doc1), (Q, Doc2), (Q, Doc3)** → positive
- **(Q, Doc10), (Q, Doc20)** → negative

### After training - how to use the encoder:

```
Qvec = Encoder(Q)           # encode query
Dvec = Encoder(Di)          # encode each document
sim(Q, Di) = Qvec · Dvec    # compute similarity (dot product)
# Sort by similarity → top-k documents
```

### Key things to remember:

- Pseudo relevance feedback → **no labels needed** → assume top-k = positive
- Hard negatives = documents just below top-k → most useful for training
- Contrastive learning = **push positives closer, push negatives further**
- After training → encode query and docs → rank by similarity

### Q1 - PageRank

- $PR(u) = \lambda/N + (1-\lambda) \times \sum PR(v)/L_v$
- **Bu** = pages pointing TO page u
- **Lv** = outgoing links FROM page v
- More inlinks → higher PR → B had highest PR (0.4625)
- No inlinks → only gets base score  $\lambda/N$  ( $C = 0.0375$ )

### Q2 - Attention to BERT

- `model.embeddings` → initial token embeddings (no context)
- `attention_mask` → 1=attend, 0=ignore padding
- `model.encoder` → passes through transformer layers
- `last_hidden_state` → final contextualised representations
- Same shape [1, 10, 768] but values change after encoder

### Q3 - Contrastive Learning + Pseudo Relevance Feedback

- Step 1: top-k retrieved docs = **positive samples**
- Step 2: random/low-ranked docs = **negative samples**
- Step 3: train encoder → increase  $\text{sim}(Q, D+)$ , decrease  $\text{sim}(Q, D-)$
- Hard negatives = just below top-k → most useful
- After training → encode → rank by dot product similarity